Study Case: URL Shortener

This study case presents a minimal two-table schema for a URL-shortener service, together with functional requirements and example workflows. The goal is to accept a long URL, produce a short code (and corresponding short URL), and redirect requests for that short URL back to the original long URL, while optionally logging each redirect event.

Functional Requirements

1. **Create Short URL**
   - **Input:** A long URL (for example, `https://www.example.com/long/path?with=params`).
   - **Output:** A unique short code (for example, `Ab3xYz`) appended to a base domain (for example, `https://sho.rt/Ab3xYz`).
   - **Behavior:**
     - Store the mapping `short_code` ↔ `long_url` with a creation timestamp.
     - Optionally, if the same long URL has already been shortened, return the existing code (or generate a new one depending on business rules).
2. **Redirect to Long URL**
   - **Input:** A short code (e.g., browser requests `GET https://sho.rt/Ab3xYz`).
   - **Behavior:**
     - Look up the long URL corresponding to that short code.
     - If found and not expired, issue an HTTP 301/302 redirect to the original long URL.
     - If not found or expired, return HTTP 404 (Not Found).
3. **Basic Analytics**
   - Record each time a short URL is accessed:
     - `mapping_id`, timestamp, client IP, user agent, etc.
   - Enable counting total clicks or unique clicks per short code.

To satisfy exactly two tables, we designate:

- `url_mappings` for all short-code ↔ long-URL pairs.
- `url_clicks` for logging each redirect event.

Table: `url_mappings`

This table stores every mapping from a generated short code to its original long URL, along with metadata such as creation time, optional expiration, and optional creator ID.

Table: `url_clicks`

Each redirect (click) of a short URL is logged in this table, capturing when it happened and optionally who clicked.

Example Workflow

1. Shortening a URL

1. Client sends:

```
POST /shorten
Content-Type: application/json

{ "long_url": "https://www.example.com/path/to/page" }
```

2. Backend (pseudocode):

```
-- 1. (Optional) Check if long_url already exists:
SELECT id, short_code
  FROM url_mappings
 WHERE long_url = 'https://www.example.com/path/to/page';

-- 2. If exists, return existing short_code. Else generate a new one:
INSERT INTO url_mappings(short_code, long_url)
VALUES ('Ab3xYz', 'https://www.example.com/path/to/page')
RETURNING id, short_code;

-- 3. Return JSON response:
{ "short_url": "https://sho.rt/Ab3xYz", "short_code": "Ab3xYz" }
```

2. Redirecting a Short URL

1. Client requests:

```
GET /Ab3xYz
```

2. Backend (pseudocode):

```
-- 1. Look up the mapping, ensuring it is not expired:
SELECT id, long_url
  FROM url_mappings
 WHERE short_code = 'Ab3xYz'
   AND (expires_at IS NULL OR expires_at > now());

-- 2. If found, let mapping_id = id, destination = long_url.
-- 3. Log the click:
INSERT INTO url_clicks(mapping_id, ip_address, user_agent)
VALUES (mapping_id, '203.0.113.45', 'Mozilla/5.0 (...)');

-- 4. Send redirect response:
HTTP/1.1 302 Found
Location: https://www.example.com/path/to/page

-- 5. If not found or expired, return 404 Not Found.
```

Additional Notes

- To enforce a one-to-one mapping for each `long_url`, you can add a unique constraint on `long_url` in `url_mappings`:

  ```
  ALTER TABLE url_mappings
    ADD CONSTRAINT uq_long_url UNIQUE (long_url);
  ```

- If analytics are not needed, you may drop `url_clicks` entirely and simply perform lookups in `url_mappings` at redirect time.
- Adjust the `expires_at` logic as needed: you could periodically archive or delete expired mappings, or check for expiration at redirect time.
- The length of `short_code` (VARCHAR(10)) accommodates up to 8 character base62 strings with room for future growth. If you choose a different encoding or length, adjust accordingly.